

Week03 | 实验 | 双源采集最小链路：从 manifest 到 raw zone 的最小闭环

本页结构

先把 Week03 的最小采集闭环跑通，再谈更复杂的流式世界	2
这次实验要完成什么	2
参考学习时间	2
学完这次实验，你应该能做到什么	2
本次实验产出	3
这次实验真正验证哪几条不变量	3
先看一张“验证证据清单”图	4
先把实验页和作业页的边界分开	4
先看一张闭环图	5
0. 这次实验为什么不额外发明新脚本	5
1. 先确认你会用到哪些 repo 对象	5
1.1 数据契约	5
1.2 清单与 schema	5
1.3 最小 ingest 入口	6
1.4 最小资产化视角	6
1.5 当前 contract tests	6
2. Step 0: 启动环境并确认健康	6
3. Step 1: 先跑 contract baseline	6
这一步你到底在验证什么	7
实验记录建议	7
4. Step 2: 跑一遍 seed_loader dry-run	7
这一步你应该关注什么	7
你应该记录下来	7
5. Step 3: 生成一份 synthetic ticket 数据，并做 ticket_ingest dry-run	7
这里最重要的不是“有没有写进库”	8
你要记录什么	8
6. Step 4: 跑一遍 document ingest dry-run	8
为什么这里也建议先 dry-run	9
这一步你要关注什么	9
建议记录	9
7. Step 5: 去 Dagster UI 看懂当前 ingest asset graph	9
你现在要建立的判断	10
你要记录什么	10
8. Step 6: 做一次“最小故障判断”，而不是假装已经自动补数	10
场景 A	10
场景 B	10
场景 C	10
为什么这一步重要	10

9. 这次实验最容易做错的 4 件事	11
错误 1: 把 contract test 当成 ingest 成功	11
错误 2: 把 dry-run 当成“不重要”	11
错误 3: 看到失败就默认“全量重跑”	11
错误 4: 实验里又造第二套路径	11
10. 你最终应该提交什么实验结果	11
这三份文件分别回答什么	11
11. 本实验最重要的 8 个判断	11
12. 自检清单	12
13. 课后最小行动	12
延伸阅读	12

先把 Week03 的最小采集闭环跑通，再谈更复杂的流式世界

这次实验不是在本本地再造一套 Kafka / Debezium / CDC 平台，而是把 Week03 已经讲过的 ingest、manifest、checkpoint、asset graph 和恢复判断，真正映射到 OmniSupport Copilot 当前 repo 的可验证对象。

你这一轮要跑出来的，不是另一套 demo，而是一条能被解释、能被记录、能自然接到作业页和 Week04 的最小采集闭环。

这次实验要完成什么

到这里，Week03 的 5 个课时已经把采集链路的关键判断都讲完了：

- 为什么 ingest 可靠性比“先把数据搬上来”更重要
- 为什么 batch 是学生本地最稳的起点
- 为什么 incremental / CDC 不能轻易承诺 exactly-once
- 为什么 asset / partition / backfill 是 ingest 的资产化视角
- 为什么 replay / rerun / backfill / runbook 必须分开讲

这次实验不再新增另一套练习世界。你要做的，是直接围绕 OmniSupport Copilot 当前 repo 已经存在的最小 ingest 基线，跑出一条真正可解释的 Week03 闭环：

contract tests → manifest baseline → ticket ingest dry-run → doc ingest dry-run → asset graph 观察 → recovery decision

参考学习时间

90—120 分钟

学完这次实验，你应该能做到什么

完成这次实验后，你应该能做到：

1. 说清楚 Week03 的 repo baseline 是什么，而不是抽象地说“我们有 ingest”。
2. 跑通 manifest、contract tests、ticket ingest dry-run、doc ingest dry-run。
3. 看懂当前 Dagster ingest asset graph 在表达什么。
4. 用实验结果判断：下一步更适合 rerun、replay 还是 backfill。
5. 产出一份能带进作业页与 Week04 的实验总结。

本次实验产出

完成实验后，请至少产出这些文件：

- docs/blueprints/week03/lab_observation_log.md
- reports/week03/lab_execution_summary.md
- reports/week03/recovery_decision_log.md

如果你愿意再多做一步，可以补：

- docs/blueprints/week03/dual_source_ingest_followups.md

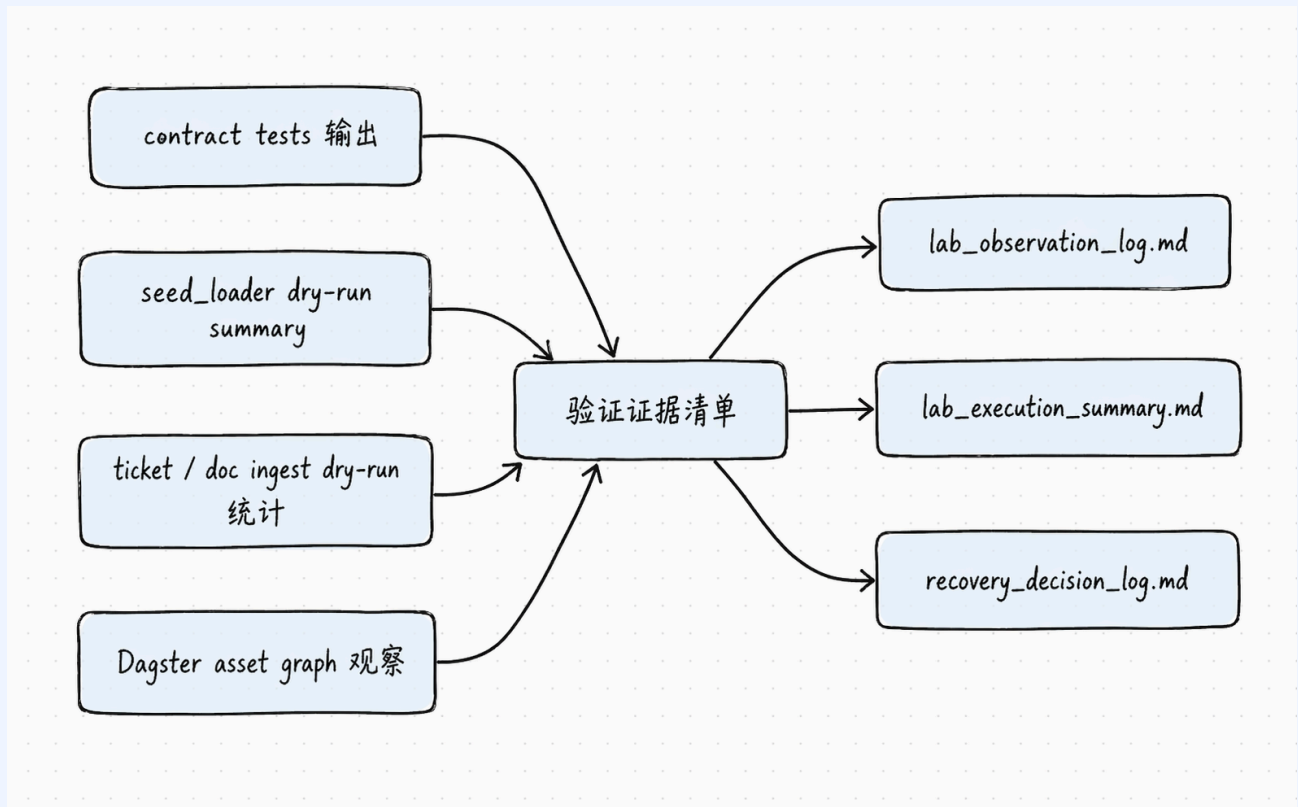
这次实验真正验证哪几条不变量

这次实验不是为了证明“命令都能跑”，而是为了验证 Week03 最小采集闭环里几条最重要的不变量。

不变量	你在实验里通过什么来验证
contract 与 manifest 是一致的	<code>pytest tests/contract/ -v + seed_loader</code> 结构校验
manifest 真的是一次 ingest 的运行声明	<code>seed_loader</code> dry-run summary 与 manifest coverage
ticket / document 两条链路都能被独立解释	<code>ticket_ingest</code> 与 <code>doc_ingest</code> 的 dry-run 结果分开记录
资产图不是装饰，而是后续 asset flow 的观察入口	Dagster UI 中的 ingest assets 与当前 repo 对象对齐
恢复动作不是拍脑袋	<code>recovery_decision_log.md</code> 中写清 rerun / replay / backfill 的选择依据

如果这些不变量没有被验证，这次实验就还只是“跑过几条命令”，还不能算 Week03 的有效实验。

先看一张“验证证据清单”图



这张图要表达的是：

- 实验页不是“把命令抄一遍”
- 而是把每一类验证证据收集起来，证明这条最小闭环真的站住了
- 如果最后没有证据清单，实验页就很容易退化成“我跑过了，但我说不清验证了什么”

先把实验页和作业页的边界分开

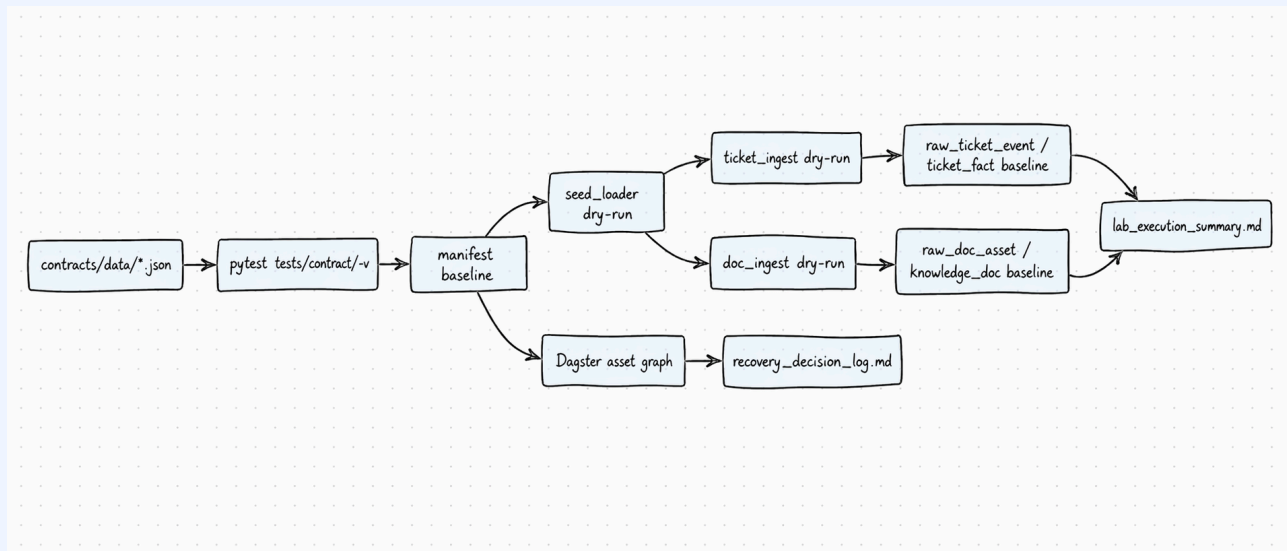
页面 这一页最核心的目标

实验页 证明最小闭环真的能被验证，弄清每一步在验证哪一层

作业页 把实验结果收口成正式交付包，供别人接手与复核

所以实验页更像 **验证与观察**，作业页更像 **整理与交付**。

先看一张闭环图



这张图就是本实验的主线：

- 先验证 **contract** 和 **manifest** 是否站得住
- 再验证 **ticket** 与 **document** 两条 ingest baseline
- 再把结果整理成 本周实验报告 + 恢复判断

0. 这次实验为什么不额外发明新脚本

这一点先讲清楚，非常重要。

当前 OmniSupport Copilot 的 README 已经把 Week01—Week03 的学生基线写得很明确：

- Week01 默认走 **Docker-only** 路线
- `seed_loader` 已经是 manifest 驱动采集框架
- `pytest tests/contract/ -v` 是最小契约验证入口
- Week03 要接入真实 MinIO 上传和 PostgreSQL 写入，而不是另起一套平行 ingest 世界

因此，这次实验的原则是：

优先复用 repo 里已经存在的命令、**contract**、**manifest** 和 ingest pipeline。

这意味着本实验不会再要求你去跑另一套 `plan_ingest.py` / `run_input_gate.py` 世界。你要练的是 **如何读懂并验证当前项目的真实基线**。

1. 先确认你会用到哪些 repo 对象

在开始之前，请先把这些对象在仓库里定位出来：

1.1 数据契约

- `contracts/data/ticket_contract.json`
- `contracts/data/doc_asset_contract.json`
- `contracts/data/audio_asset_contract.json`
- `contracts/data/video_asset_contract.json`

1.2 清单与 schema

- `data/seed_manifests/manifest_tickets_synthetic_v1.json`

- data/seed_manifests/manifest_workspace_helpcenter_v1.json
- data/seed_manifests/manifest_edge_gateway_pdf_v1.json
- data/seed_manifests/source_manifest_schema.json

1.3 最小 ingest 入口

- pipelines/ingestion/seed_loader.py
- pipelines/ingestion/ticket_ingest.py
- pipelines/ingestion/doc_ingest.py

1.4 最小资产化视角

- pipelines/ingestion/assets.py
- pipelines/definitions.py

1.5 当前 contract tests

- tests/contract/test_json_schemas.py

先把这些对象找齐，再开始跑命令。因为这次实验的重点，不只是“命令跑过”，而是知道你到底在验证哪一层。

2. Step 0: 启动环境并确认健康

先按 README 的 Week01–Week03 基线启动环境。

```
cp infra/env/.env.example infra/env/.env.local
```

```
docker compose \
  --env-file infra/env/.env.local \
  -f infra/docker-compose.yml \
  up -d --build
```

然后确认健康：

```
curl http://localhost:8000/health
curl http://localhost:8001/health
```

浏览器入口也建议先记下来：

- Dagster UI: <http://localhost:3000>
- MinIO Console: <http://localhost:9001>
- Phoenix: <http://localhost:6006>

如果这一步没有通过，后面所有实验都不应该继续。

3. Step 1: 先跑 contract baseline

先执行：

```
docker compose \
  --profile tools \
  --env-file infra/env/.env.local \
  -f infra/docker-compose.yml \
  run --rm devbox \
  pytest tests/contract/ -v
```

这一步你到底在验证什么

你不是在验证“所有 ingest 都成功了”，而是在验证：

- 四类 contract 文件存在
- JSON Schema 至少合法
- ticket contract 关键字段存在
- seed manifests 与 `source_manifest_schema.json` 相容

也就是说，这一步是 **Week02 contract 思维** 进入 **Week03 ingest** 之前的第一道门。

实验记录建议

在 `docs/blueprints/week03/lab_observation_log.md` 里先写下：

- 哪些 contract 被检查了
- 哪些 manifest 被检查了
- 当前 contract baseline 是否全部通过
- 如果失败，失败在 schema 还是文件缺失

4. Step 2：跑一遍 seed_loader dry-run

接着执行：

```
docker compose \
  --profile tools \
  --env-file infra/env/.env.local \
  -f infra/docker-compose.yml \
  run --rm devbox \
  python -m pipelines.ingestion.seed_loader \
  --manifest-dir data/seed_manifests
```

这一步你应该关注什么

`seed_loader.py` 当前承担的是 **manifest 驱动采集框架** 的角色。它会做两类校验：

1. 结构校验
 - 通过 `source_manifest_schema.json` 检查 manifest 结构是否合法
2. 业务校验
 - 检查关键业务字段，比如 `license_tag`、`pii_level` 等

你此时要建立的判断是：

manifest 不是“文件列表”，而是一次 ingest 的最小运行声明。

你应该记录下来

在 `reports/week03/lab_execution_summary.md` 里记一段：

- 本次 `seed_loader` 读到了哪些 manifest
- 是单一模态，还是多模态混合
- 是否出现了 schema violation 或关键字段缺失
- 如果 manifest 出问题，它属于 Week02 的 contract 问题，还是 Week03 的 ingest 问题

5. Step 3：生成一份 synthetic ticket 数据，并做 ticket_ingest dry-run

先生成 synthetic tickets：

```
docker compose \
  --profile tools \
  --env-file infra/env/.env.local \
  -f infra/docker-compose.yml \
  run --rm devbox \
  python data/synthetic_generators/ticket_simulator.py \
  --count 200 \
  --output data/canonization/tickets/tickets-lab-001.jsonl
```

然后运行 dry-run:

```
docker compose \
  --profile tools \
  --env-file infra/env/.env.local \
  -f infra/docker-compose.yml \
  run --rm devbox \
  python -m pipelines.ingestion.ticket_ingest \
  --input data/canonization/tickets/tickets-lab-001.jsonl \
  --batch-id lab-week03-ticket-001 \
  --dry-run
```

这里最重要的不是“有没有写进库”

因为 dry-run 下，核心目的是让你先看懂：

- ticket_ingest.py 如何吃 JSONL
- 它怎样复用 ticket_contract.json
- 它如何统计 total / valid / invalid / skipped / errors
- 为什么 batch_id 从 Week03 开始就必须认真对待

ticket_ingest.py 本身已经把最小批处理骨架写出来了：

- 读 JSONL
- 做 JSON Schema 校验
- 做业务规则校验
- 生成 summary

这正是 Week03 课时 2 里讲的“批处理主链路”的最小可运行形态。

你要记录什么

在 reports/week03/lab_execution_summary.md 中补下面这些项：

- 输入文件路径
- batch_id
- total / valid / invalid / skipped / errors
- dry-run 下你观察到的 summary
- 哪些字段是 contract 验证直接拦住的
- 哪些错误更像业务规则错误，而不是 schema 错误

6. Step 4: 跑一遍 document ingest dry-run

接着执行：


```
docker compose \
  --profile tools \
  --env-file infra/env/.env.local \
  -f infra/docker-compose.yml \
  run --rm devbox \
  python -m pipelines.ingestion.doc_ingest \
  --manifest data/seed_manifests/manifest_workspace_helpcenter_v1.json \
  --dry-run \
  --batch-id lab-week03-doc-001
```

为什么这里也建议先 dry-run

因为 doc_ingest.py 当前已经包含了：

- manifest 校验
- source asset 遍历
- MinIO 上传分支
- PostgreSQL 元数据写入分支
- 文档 summary 统计

而 dry-run 的价值在于：你可以先看清楚路径、元数据与对象落点，而不用马上把注意力放在对象存储和数据库副作用上。

这一步你要关注什么

对文档 ingest，最重要的是看：

- manifest 是如何约束 document assets 的
- source_url_or_path 如何进入 pipeline
- source_fingerprint、license_tag、product_line 等字段为什么重要
- 为什么 Week03 只是把文档先推进到 raw zone 与 metadata baseline，而不是直接做完整解析和向量化

建议记录

继续在 reports/week03/lab_execution_summary.md 里补：

- 本次 manifest 处理的 document source 是什么
- total / uploaded / skipped / db_inserted / errors 的 summary
- 你认为这条链路最需要补强的是：
 - 文件可达性
 - source fingerprint
 - 元数据完整性
 - MinIO / DB 副作用验证

7. Step 5: 去 Dagster UI 看懂当前 ingest asset graph

打开：

<http://localhost:3000>

然后重点看当前定义的 ingestion 资产：

- seed_manifests
- raw_doc_assets
- raw_ticket_events

- `ingest_all_job`

你现在要建立的判断

当前 repo 里的 Dagster 还不是“全自动的生产编排系统”，它更像是 **Week03** 的资产化骨架。

也就是说：

- 它已经把“manifest → Bronze ingest”的方向表达出来了
- 但很多写入逻辑、回放逻辑、恢复逻辑还处在逐步接入阶段

这正是为什么课时 4 会反复强调：

manifest 想做什么，asset 表达会产生什么，job 只是触发它们的方式。

你要记录什么

在 `docs/blueprints/week03/lab_observation_log.md` 里记：

- 当前 asset graph 已经表达了哪些 ingest 关系
- 哪些 asset 还只是骨架
- 如果 Week04 要接 Iceberg Bronze / Silver，哪几个 asset 会最先演化

8. Step 6：做一次“最小故障判断”，而不是假装已经自动补数

这次实验不要伪造一个不存在的 replay engine。更成熟的做法是：

先训练你对恢复动作的判断。

请你在 `reports/week03/recovery_decision_log.md` 里回答下面 3 个场景：

场景 A

manifest schema 通过了，但某个 ticket batch 有 12% 记录 invalid。你更倾向：– rerun – replay – backfill – 先隔离并修 contract / source data

场景 B

document ingest dry-run 通过，但 MinIO 上传阶段失败。你更倾向：– 重新执行这次 batch – 回放 manifest – 回滚 contract – 人工核对 source_dir

场景 C

一周后发现一批旧 ticket 没有进入 `ticket_fact`。你更倾向：– 直接全量重跑 – 做 targeted backfill – 先对齐 checkpoint / batch_id，再决定 replay

为什么这一步重要

因为 Week03 当前 repo 的现实边界是：

- ingest baseline 已有
- manifest / contract / dry-run 已有
- 资产图骨架已有

但完整的自动 replay / backfill engine 还没有被完全实现成学生本地一键工具。所以在这一阶段，你更需要的是：

恢复思维 + Runbook 思维 + 边界判断。

9. 这次实验最容易做错的 4 件事

错误 1：把 contract test 当成 ingest 成功

不是。

contract test 只说明： – schema / key fields / manifest shape 还站得住

它不等于： – 数据已经写入成功 – side effect 已经完成 – downline 已经可消费

错误 2：把 dry-run 当成“不重要”

也不是。

dry-run 是 Week03 最关键的中间层： – 它能帮你先确认运行声明、路径、summary 和批次边界 – 而不让你一上来就被对象存储和数据库副作用干扰

错误 3：看到失败就默认“全量重跑”

这是课时 5 反复纠正的思维。

真正成熟的动作是先判断： – 是 source data 问题 – 是 contract 问题 – 是 manifest 问题 – 还是 side effect 执行问题

错误 4：实验里又造第二套路径

不需要。

你现在要做的，是围绕当前 repo 基线建立理解和证据，而不是再开一套新目录把问题藏起来。

10. 你最终应该提交什么实验结果

建议最少交这 3 份：

- docs/blueprints/week03/lab_observation_log.md
- reports/week03/lab_execution_summary.md
- reports/week03/recovery_decision_log.md

这三份文件分别回答什么

文件	它回答的问题
lab_observation_log.md	当前 ingest baseline 在 repo 里长什么样
lab_execution_summary.md	本次 contract / manifest / ticket / doc 实验各跑出了什么
recovery_decision_log.md	如果下一步真的进 Week04 / Week05，故障恢复你会怎么判断

11. 本实验最重要的 8 个判断

1. Week03 的实验不是搭新系统，而是读懂并验证当前 repo 的 ingest baseline。
2. contract tests、seed loader、ticket ingest、doc ingest 各自验证的层次不同。
3. dry-run 不是“偷懒”，而是控制副作用、先看清链路的工程习惯。
4. batch_id、manifest_id、source_fingerprint 这些锚点不是装饰字段，而是恢复与追踪的基础。
5. 资产图骨架已经存在，但自动 replay / backfill 仍需要更后面的实现推进。
6. 当前阶段最成熟的学习目标，不是“一键补数”，而是“知道什么时候该 replay、什么时候该 rerun、什么时候该 backfill”。

7. 实验结果不是课堂笔记，而是可以直接带入作业页和 Week04 的工程资产。
8. 这次实验的价值，在于把 Week03 的抽象判断变成 repo 里的可验证对象。

12. 自检清单

在进入作业前，请至少确认这些项：

- ☐ 我已经跑过 `pytest tests/contract/ -v`
- ☐ 我已经跑过 `seed_loader baseline`
- ☐ 我已经跑过 `ticket_ingest --dry-run`
- ☐ 我已经跑过 `doc_ingest --dry-run`
- ☐ 我已经看过 Dagster UI 里的最小 ingest assets
- ☐ 我已经写出三份实验记录
- ☐ 我能解释哪一步在验证 schema、哪一步在验证 manifest、哪一步在验证 batch ingest
- ☐ 我能说清楚当前 repo 的 replay / backfill 边界在哪里

13. 课后最小行动

如果你这次实验时间不够，至少做完下面四步：

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run --rm devbox pytest tests/contract/ -v
```

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run --rm devbox \
    python -m pipelines.ingestion.seed_loader --manifest-dir data/seed_manifests
```

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run --rm devbox \
    python data/synthetic_generators/ticket_simulator.py --count 200 --output data/canonization/tickets/tickets-lab-001.jsonl
```

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run --rm devbox \
    python -m pipelines.ingestion.ticket_ingest --input data/canonization/tickets/tickets-lab-001.jsonl --batch-id lab-week03-ticket-001 --dry-run
```

然后把你的观察写进：

- docs/blueprints/week03/lab_observation_log.md
- reports/week03/lab_execution_summary.md

延伸阅读

- OmniSupport Copilot README (Docker-first / Week03 baseline)
- pipelines/ingestion/seed_loader.py
- pipelines/ingestion/ticket_ingest.py
- pipelines/ingestion/doc_ingest.py
- pipelines/ingestion/assets.py
- tests/contract/test_json_schemas.py